arguments and returns void. And then the actual debounce function declares a generic function extending Debounceable and says that both the argument function and the return function have that type, which specifies that the return value of debounce has the same signature as the argument function, which is what we want. The result is a little verbose, to my eyes, but is more type safe than before.

The issues in the CalendarController seem legit to me; it's a failure of the code to deal with the case where the target.dataset.scheduleAttribute doesn't exist. So either we need to explicitly say it could be undefined, or we need to provide a default value. I think the thing to do is allow the argument to be undefined, but then guard against it:

chapter_13/04/app/javascript/controllers/calendar_controller.ts

```
filter(): void {
  const everyDayUnselected = this.everyDayUnselected()
  this.calendarDayTargets.forEach((target: HTMLElement) => {
    const show =
      everyDayUnselected || target.dataset.cssStatusValue === "true"
    this.toggleAssociatedConcerts(target.dataset.scheduleAttribute, !show)
  })
}

showAll(): void {
  this.calendarDayTargets.forEach((target: HTMLElement) => {
    target.dataset.cssStatusValue = "false"
    this.toggleAssociatedConcerts(target.dataset.scheduleAttribute, false)
  })
}

toggleAssociatedConcerts(
  attributeName: string | undefined,
  toggleValue: boolean
): void {
  if (!attributeName) {
    return
  }
  document
    .querySelectorAll(`.concert[${attributeName}]`)
    .forEach((element) => {
      element.classList.toggle("hidden", toggleValue)
```